

Employee Attrition Prediction

A Machine Learning Approach to Identifying Flight-Risk Employees

Course: CSE422 — Artificial Intelligence

Project Type: Lab Project Report

Dataset: employee_attrition_dataset.csv (1,677 employee records)

Tools: Python, pandas, scikit-learn, seaborn, matplotlib

Submitted as part of the CSE422 Lab coursework

Table of Contents

1. Introduction	4
2. Dataset Description	5
2.1 Dataset Overview	5
2.2 Classification or Regression Problem?	5
2.3 Feature Types: Quantitative vs. Categorical	5
2.4 Do the Categorical Variables Need Encoding?	5
2.5 Correlation of All Features	6
2.6 Imbalanced Dataset Check	9
2.7 Exploratory Data Analysis	10
3. Dataset Pre-processing	13
Problem 1 & Solution — Useless Columns	13
Problem 2 & Solution — Invalid / Hidden Missing Values	13
Problem 3 & Solution — Categorical Values	14
Problem 4 & Solution — Feature Scaling	14
Pre-processing Summary	15
4. Dataset Splitting	16
5. Model Training & Testing	17
5.1 Logistic Regression	17
5.2 Naive Bayes	18
5.3 Neural Network (Multi-Layer Perceptron)	18
5.4 Unsupervised Approach — K-Means Clustering	18
6. Model Selection / Comparison Analysis	21
6.1 Prediction Accuracy Comparison	21
6.2 Precision and Recall Comparison	21
6.3 Confusion Matrices	22
6.4 ROC Curve and AUC Score	22
6.5 Best Model per Metric	23
6.6 The Accuracy Paradox: Imbalanced vs. Balanced Training	24
7. Conclusion	25

7.1 What We Understand From the Results 25

7.2 Comments on Each Model's Performance 25

7.3 Why We Are Getting These Results 26

7.4 Challenges Faced 26

7.5 Final Summary 27

1. Introduction

Every organisation loses employees, but losing them **unexpectedly** is what actually costs money. Replacing an employee means running interviews, onboarding a newcomer, and absorbing the lost productivity and institutional knowledge while the role sits empty or under-trained. If the Human Resources department could flag at-risk employees before they resign, it would have a real opportunity to intervene — through a workload adjustment, a compensation review, or simply a conversation — before the decision to leave becomes final.

This project treats employee attrition as a supervised machine learning problem. Given a snapshot of an employee's profile — their age, income, job role, satisfaction survey answers, overtime status, tenure, and so on — the goal is to predict whether that employee falls into the group that **left the company** (`Attrition = 1`) or the group that **stayed** (`Attrition = 0`).

The dataset used is `employee_attrition_dataset.csv`, containing 1,677 employee records across 35 original columns, including demographic information, job-related attributes, compensation figures, and four workplace satisfaction surveys. The motivation for choosing this dataset is threefold: attrition is a genuinely costly and common HR problem, the data offers a realistic mix of numerical and categorical features that requires deliberate pre-processing decisions, and the binary target makes it a clean fit for the classification techniques covered in this course.

Across this report, the dataset is explored and cleaned, three supervised classifiers (Logistic Regression, Naive Bayes, and a Neural Network) are trained and compared, and the same data is separately explored through K-Means clustering as an unsupervised learning exercise. The report closes with a comparison of all models across accuracy, precision, recall, F1-score and AUC, followed by a discussion of what the results reveal about the underlying problem.

2. Dataset Description

2.1 Dataset Overview

The dataset contains **1,677 data points (rows)**, one per employee, and **35 columns** in total. Excluding the target column, this leaves **34 input (predictor) features** describing each employee, with **Attrition** serving as the single output feature to be predicted.

Property	Value
Total data points (rows)	1,677
Total columns	35
Input (predictor) features	34
Output / target feature	Attrition (binary: 0 = stayed, 1 = left)

2.2 Classification or Regression Problem?

This is a **binary classification problem**. The target column **Attrition** takes only two discrete values — **0** (the employee stayed) or **1** (the employee left) — and there is no meaningful in-between value. A prediction of, say, 0.63 would be meaningless here, because an employee cannot partially leave the company. The task is to sort each employee into one of exactly two labelled groups, which is precisely what classification does. Had the target instead been a continuous quantity — for example, predicting an employee's **MonthlyIncome** or the number of years they are expected to stay — the appropriate approach would have been regression instead.

2.3 Feature Types: Quantitative vs. Categorical

Splitting the columns by data type gives **27 quantitative (numerical) features** and **8 categorical (text) features**. It is worth noting that not every numerical column is truly continuous: nine of the "numerical" columns (**Education**, **EnvironmentSatisfaction**, **JobInvolvement**, **JobLevel**, **JobSatisfaction**, **PerformanceRating**, **RelationshipSatisfaction**, **StockOptionLevel**, and **WorkLifeBalance**) are ordinal survey ratings stored as small integers, and behave more like ordered categories than continuous quantities.

Category	Count	Examples
Quantitative (numerical)	27	Age, MonthlyIncome, DailyRate, TotalWorkingYears, YearsAtCompany
— of which ordinal ratings	9	Education, JobSatisfaction, WorkLifeBalance, StockOptionLevel
Categorical (text)	8	BusinessTravel, Department, JobRole, MaritalStatus, OverTime

2.4 Do the Categorical Variables Need Encoding?

Yes. Scikit-learn models are fundamentally mathematical: KNN-style distance measures, Logistic Regression's weighted sums, and a Neural Network's matrix multiplications all require numeric input. None of these operations can be applied directly to a string such as "Sales Executive" or "Married" — the model would simply raise an error. Encoding converts these text labels into numbers while trying to preserve the information they carry, and the specific method chosen depends on whether the category has a natural order. The encoding strategy actually applied to each column is detailed in Section 3.

2.5 Correlation of All Features

Before pre-processing, a temporary numeric encoding was applied to the text columns purely to compute a full correlation matrix across every input and the output feature. The three constant columns (see Section 3) were excluded, since a column with zero variance has an undefined correlation. The resulting heatmap below shows the pairwise correlation of all remaining features, input and output alike.

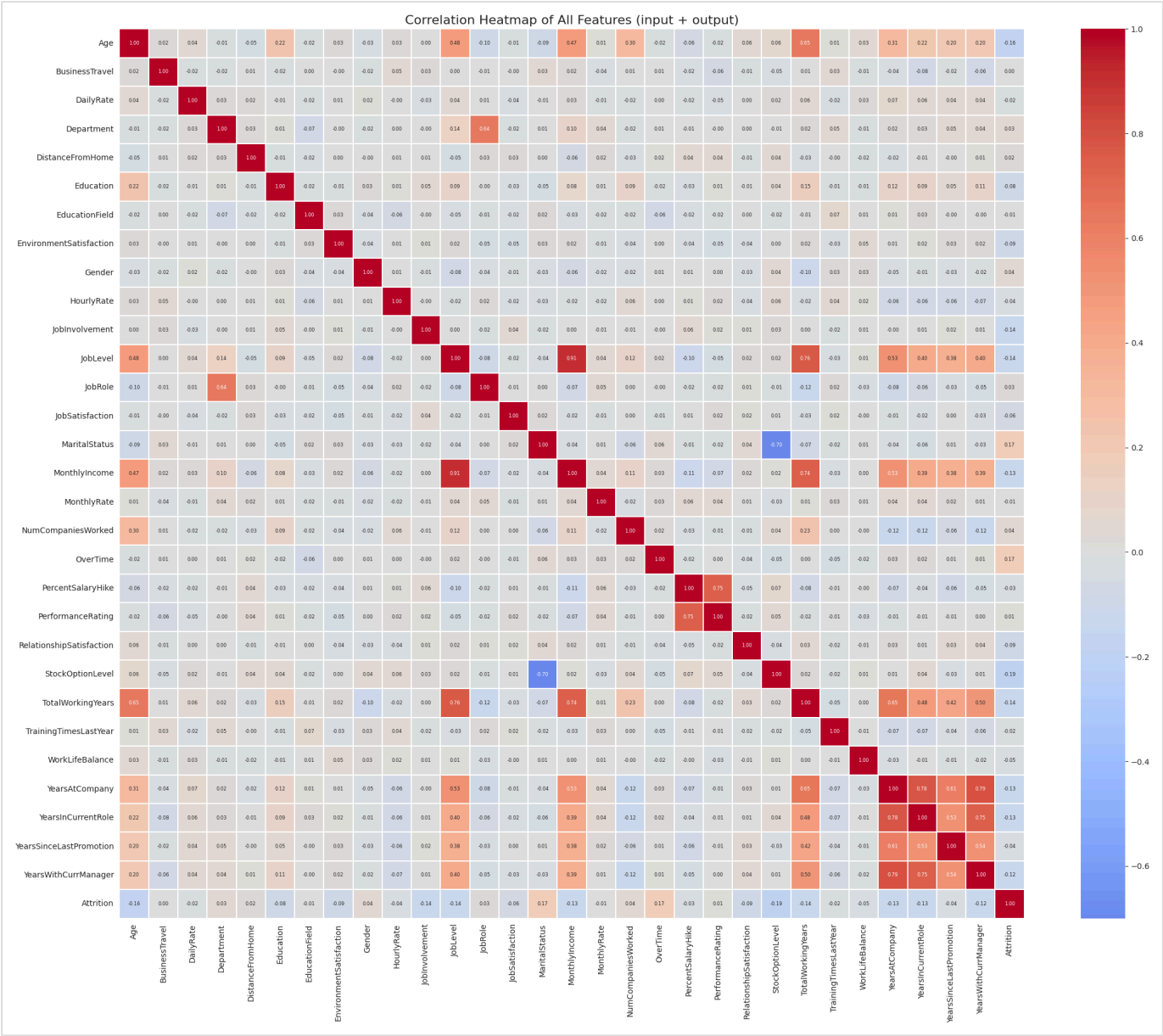


Figure 2.1 — Full correlation heatmap of all input and output features.

Because the full heatmap is dense, the single column that matters most for this project — the correlation of each input feature with the target **Attrition** — is isolated below.

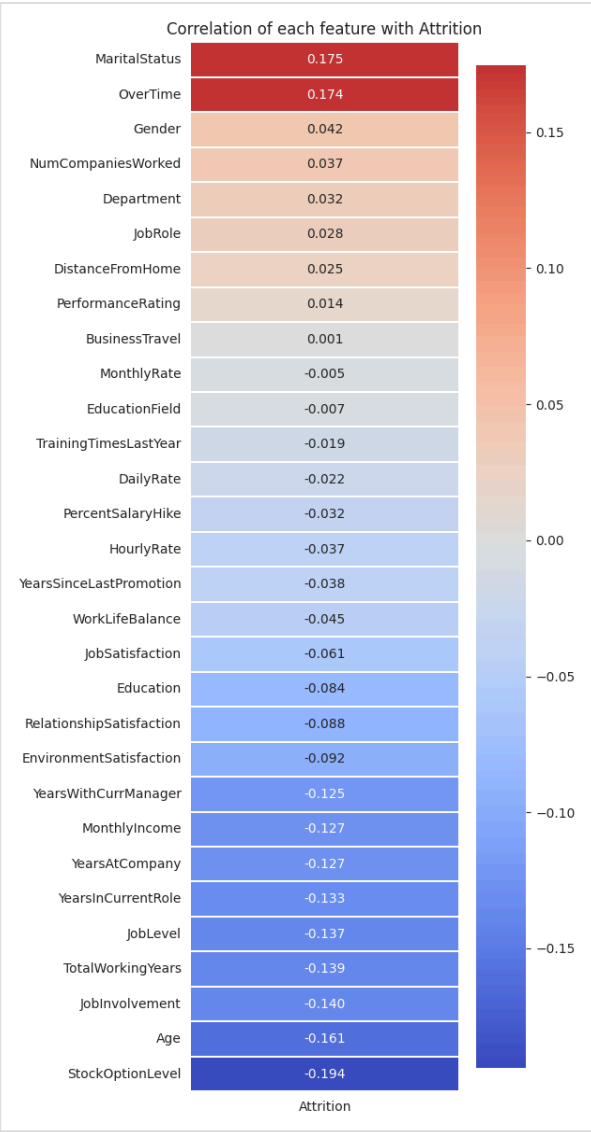


Figure 2.2 — Correlation of each individual feature with Attrition, sorted from most positive to most negative.

What We Understand After the Correlation Test

KEY OBSERVATIONS

- **No feature is strongly correlated with Attrition.** Every value lies roughly between -0.20 and $+0.18$ (`MaritalStatus` at $+0.175$ is the strongest positive value; `StockOptionLevel` at -0.194 is the strongest negative one). This means no single column can predict attrition on its own — a genuinely difficult signal for any model to learn.
- **The strongest signals still make practical sense.** `StockOptionLevel` and `MonthlyIncome` are negative, so employees who are financially invested in the company tend to stay. `Age`, `TotalWorkingYears`, and `YearsAtCompany` are also negative, so older and more settled employees leave less often. `OverTime` and `MaritalStatus` are positive, meaning overworked and single employees leave more.
- **Because the correlations are weak but spread across many columns**, most features were retained rather than dropped — the models need to combine several weak signals rather than rely on one strong one.
- **Correlation only measures straight-line (linear) relationships.** A model such as a Decision Tree or Neural Network can still exploit non-linear or categorical patterns that this test is blind to — a point that becomes very relevant in Section 2.7.

Multicollinearity: Are Any Input Features Redundant?

The correlation matrix was also checked for pairs of *input* features that are highly correlated with each other (a threshold of $|r| > 0.7$ was used):

Feature Pair	Correlation (r)
JobLevel ↔ MonthlyIncome	0.910
YearsAtCompany ↔ YearsWithCurrManager	0.792
YearsAtCompany ↔ YearsInCurrentRole	0.781
JobLevel ↔ TotalWorkingYears	0.762
PercentSalaryHike ↔ PerformanceRating	0.752
YearsInCurrentRole ↔ YearsWithCurrManager	0.751
MonthlyIncome ↔ TotalWorkingYears	0.741

These pairs are expected in HR data — a higher job level naturally comes with a higher income, and an employee with many years at the company usually has many years with the same manager. Rather than dropping one feature from each pair on the assumption that this would help, this decision was tested directly. A parallel experiment was run in which one feature from each correlated cluster (`JobLevel`, `YearsWithCurrManager`, and `YearsInCurrentRole`) was removed, and every model was retrained from scratch on the resulting 40-feature set instead of the original 43.

Model	Δ AUC	Δ F1-Score	Δ Recall	Verdict
Logistic Regression	+0.0042	+0.0029	0.000	Effectively unchanged

Naive Bayes	+0.0109	+0.0002	−0.025	Marginal gain, recall slightly worse
Neural Network	−0.0159	−0.1034	−0.125	Noticeably worse

The changes were small in every direction and, for the Neural Network, clearly negative. Naive Bayes did improve slightly, which is consistent with theory — its core assumption is that features are independent, and removing one half of a 0.91-correlated pair reduces double-counting of the same evidence. However, that gain was marginal (+0.011 AUC) and came with a drop in recall, while the Neural Network lost over ten points of F1-score, most likely because it is able to use correlated inputs jointly rather than redundantly, so the "removed" columns were still carrying useful signal for it. Logistic Regression barely moved at all, since scikit-learn applies L2 regularisation by default, which already shrinks correlated coefficients toward each other before this was ever considered.

On the strength of this comparison, **all 43 encoded features were kept** for the final models reported in this project. High correlation between two inputs affects how individual regression *coefficients* should be interpreted, but it did not meaningfully harm predictive performance here — and since the goal of this project is prediction rather than causal interpretation of individual coefficients, keeping the full feature set was the better choice.

2.6 Imbalanced Dataset Check

The output feature **Attrition** does **not** have an equal number of instances across its two classes:

Class	Meaning	Count	Percentage
0	Stayed	1,477	88.07%
1	Left	200	11.93%

This gives an imbalance ratio of roughly **7.38 : 1** (majority : minority).

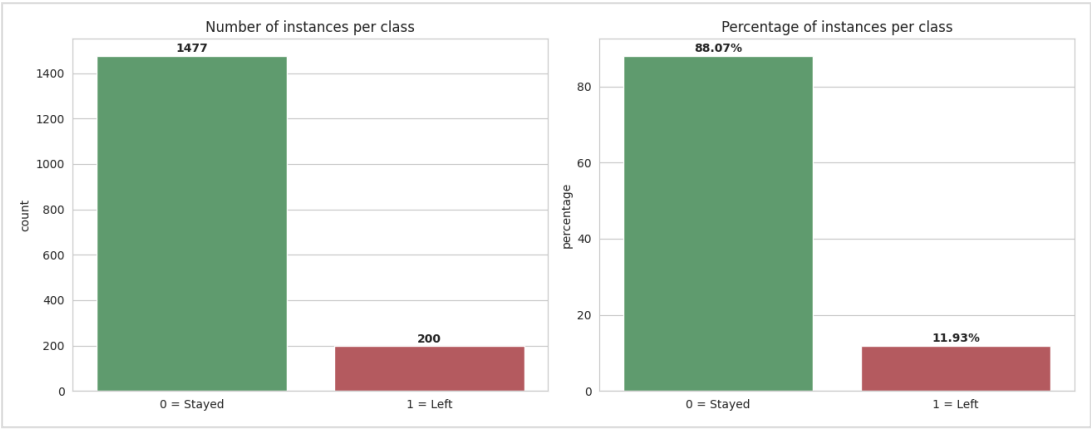


Figure 2.3 — Bar chart of the two Attrition classes, by count and by percentage.

WHY THIS MATTERS

A model can score 88% accuracy by lazily predicting "stayed" for every single employee, without learning anything at all. Such a model would be useless to HR, since it would never catch the employees who are actually about to leave — the only group that matters for this problem. Because of this, accuracy alone is a misleading headline metric here; Section 6 also reports precision, recall, F1 and AUC, and the training data is rebalanced before model fitting (Section 4).

2.7 Exploratory Data Analysis — Extracting Relationships

Beyond the linear correlation test, several relationships were explored directly by comparing categorical groups and numerical distributions against the target. This step is necessary in addition to correlation, not instead of it — correlation only captures straight-line relationships between two numeric-coded variables, and it can badly understate (or entirely miss) the effect of a categorical feature whose categories have no natural numeric order. **JobRole** is the clearest example in this dataset: its correlation with Attrition was a negligible +0.028, yet the plot below shows Sales Representatives leaving at nearly 34% while Research Directors leave at under 3% — a massive real effect that correlation, computed on arbitrary category codes, could not detect.

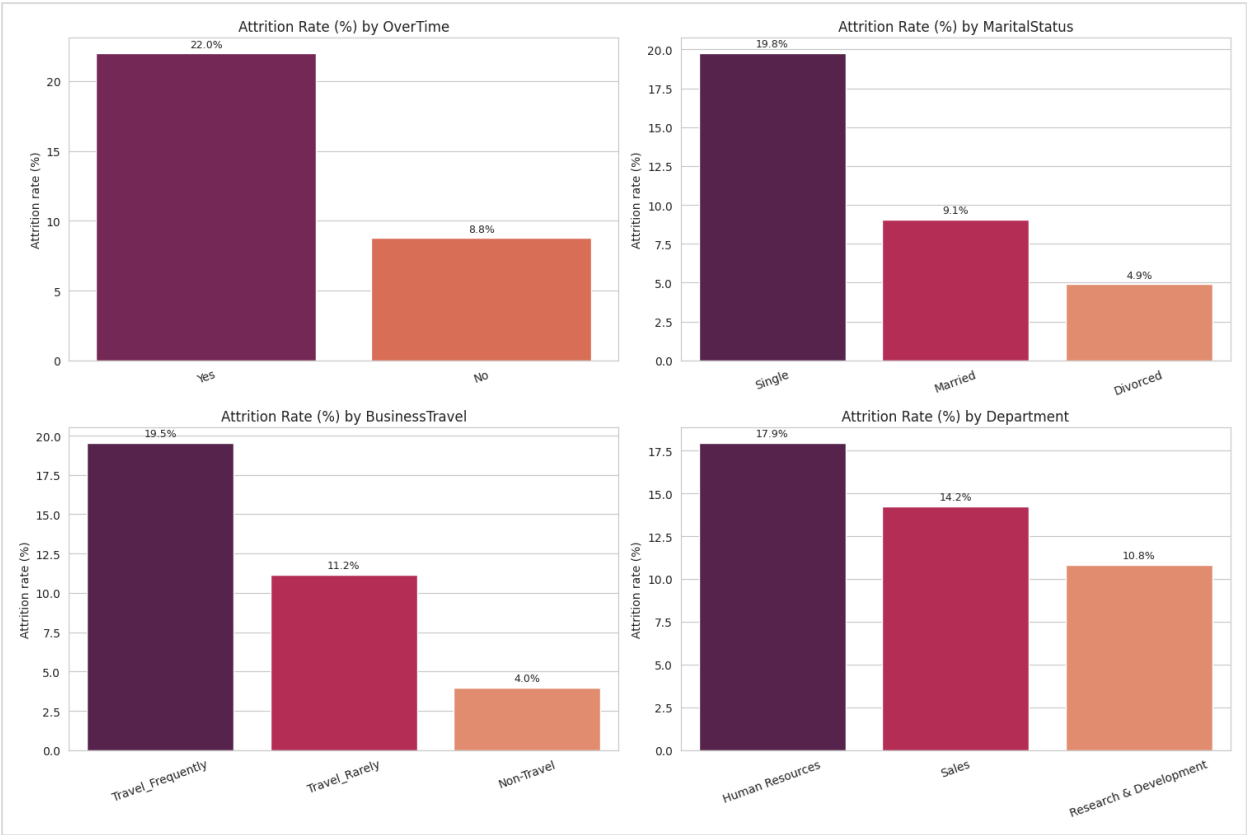


Figure 2.4 — Attrition rate by OverTime, MaritalStatus, BusinessTravel, and Department.

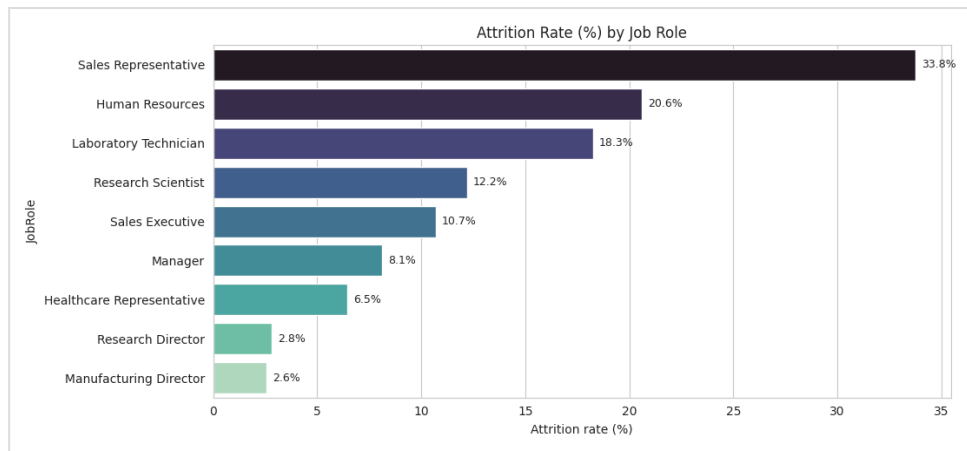


Figure 2.5 — Attrition rate by JobRole. Sales Representatives leave far more than any other role.

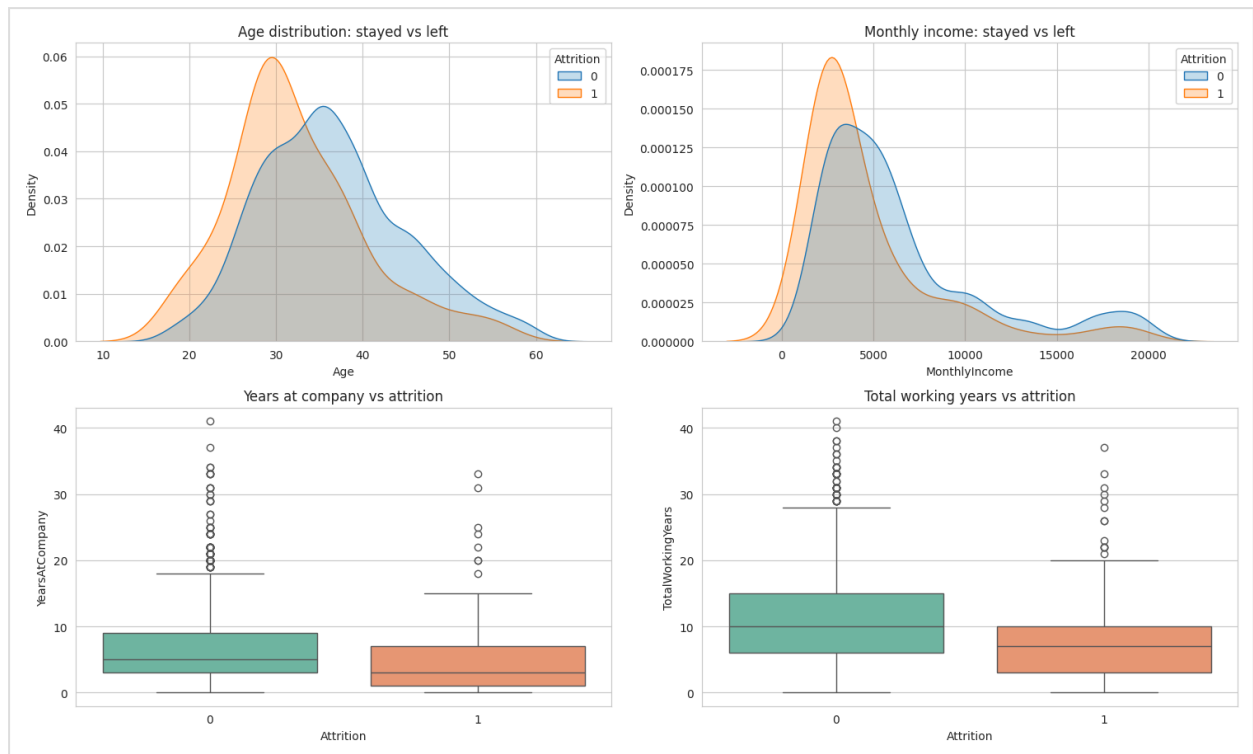


Figure 2.6 — Age and monthly income distributions, and years of tenure, split by whether the employee stayed or left.

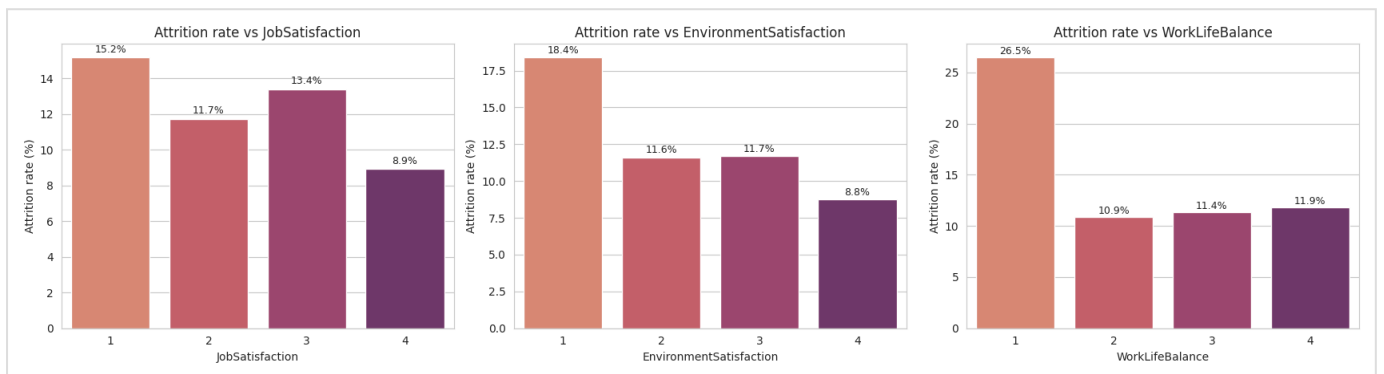


Figure 2.7 — Attrition rate against the three main satisfaction survey scores.

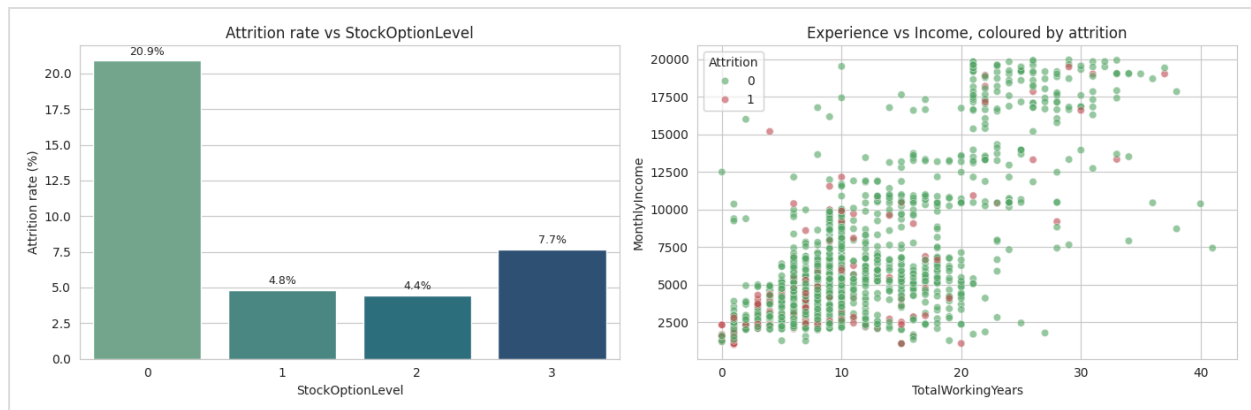


Figure 2.8 — Attrition rate by StockOptionLevel, and the relationship between experience and income coloured by attrition.

KEY RELATIONSHIPS FOUND DURING EDA

1. **Overtime is the clearest single warning sign.** Employees working overtime leave at 22.00%, versus 8.77% for those who do not — roughly a 2.5× higher rate.
2. **Money matters a lot.** The median monthly income of employees who left is 2,921.5, well below the 5,094 median of those who stayed.
3. **Stock options lock people in.** Employees at StockOptionLevel 0 leave at 20.90%, compared to roughly 4–8% for every level above 0 — about a 4× difference.
4. **Younger, newer employees are the flight risk.** The median age of leavers is 31 versus 36 for stayers; median YearsAtCompany is 3 versus 5; median TotalWorkingYears is 7 versus 10.
5. **Role matters strongly.** Sales Representatives (33.77%), Human Resources (20.59%), and Laboratory Technicians (18.26%) show the highest attrition rates of any job role.
6. **Marital status matters.** Single employees leave at 19.76%, versus 9.06% for married and 4.89% for divorced employees.

3. Dataset Pre-processing

Four distinct faults were found in the raw dataset. Each is discussed individually below, followed immediately by the solution applied to it, before moving on to the next.

Problem 1 — Useless Columns (Identifiers and Constants)

PROBLEM — COLUMNS THAT CARRY NO INFORMATION

Four columns cannot help any model:

- `id` — a row number, unique per employee. A model could memorise it instead of learning a real pattern.
- `EmployeeCount` — the value is 1 in all 1,677 rows.
- `StandardHours` — the value is 80 in all 1,677 rows.
- `Over18` — the value is 'Y' in all 1,677 rows.

A column with the same value everywhere has zero variance. It cannot separate a leaver from a stayer, since both groups share the identical value — it only adds noise and unnecessary computation.

SOLUTION — DELETE THE COLUMNS

Technique: column deletion, not row deletion. The fault affects an entire column for every row, so dropping rows would needlessly discard 1,677 perfectly good employee records to fix a column-level issue.

Dropped: `id`, `EmployeeCount`, `StandardHours`, `Over18`. Shape changed from (1677, 35) to (1677, 31).

Problem 2 — Invalid Values Hiding as Normal Numbers

PROBLEM — IMPOSSIBLE VALUES WITH NO BLANK CELLS

A direct `df.isnull().sum()` check reported **zero** missing cells, so the dataset appears perfectly clean at first glance. It is not. Checking each rating column against its documented valid range surfaced hidden data-entry errors:

Column	Valid Range	Invalid Value Found
Education	1–5	15
JobLevel	1–5	7
DailyRate	1–1499	3921

In addition, one row had `TotalWorkingYears > Age`, and two rows had `YearsAtCompany > TotalWorkingYears` — both logically impossible for a real person.

SOLUTION — FLAG AS MISSING, THEN IMPUTE WITH THE MEDIAN

Technique: convert each impossible value to **NaN** (honestly marking it as missing), then fill the resulting gaps using the **median** of that column — not the mean, since the mean would itself be dragged upward by the very outliers being corrected, while the median lands on a realistic value already consistent with the survey scale. Only a handful of rows were affected, and given that the minority (Left) class has just 200 rows in total, imputing rather than deleting preserves data that this small dataset cannot afford to lose.

The two rows where `YearsAtCompany` exceeded `TotalWorkingYears` were capped rather than deleted, by setting `YearsAtCompany` equal to `TotalWorkingYears` for those rows. Result: 3 rows fixed, 0 illogical rows remaining, shape unchanged at (1677, 31).

Problem 3 — Categorical (Text) Values

PROBLEM — TEXT COLUMNS CANNOT BE USED BY ANY MODEL

Seven text columns remained after Problem 1 (`Over18` already dropped): `BusinessTravel`, `Department`, `EducationField`, `Gender`, `JobRole`, `MaritalStatus`, `OverTime`. Scikit-learn models cannot perform arithmetic on a string such as "Sales Executive" — KNN-style models need a distance, Logistic Regression needs a weighted sum, and a Neural Network needs matrix multiplication. Feeding text directly would raise an error.

SOLUTION — ENCODE EACH COLUMN WITH A METHOD SUITED TO ITS CATEGORY TYPE

A single encoding method for every column would be incorrect — the right choice depends on whether the categories have a natural order:

- **Binary columns** → **map directly to 0/1**. `Gender` (Male=1, Female=0) and `OverTime` (Yes=1, No=0). Only two options exist, so a single 0/1 column captures everything.
- **Ordinal column** → **ordered integers**. `BusinessTravel` (Non-Travel=0, Travel_Rarely=1, Travel_Frequently=2). The order is genuine — travelling frequently really is "more travel" than travelling rarely.
- **Nominal columns** → **One-Hot Encoding**. `Department`, `EducationField`, `JobRole`, `MaritalStatus`. These have no natural order; a plain integer code would wrongly imply that "Manager" is numerically "greater than" or "less than" another role. One-Hot creates a separate 0/1 column per category instead. `drop_first=True` was used to avoid the dummy-variable trap, where one column becomes perfectly predictable from the rest and destabilises linear models.

After encoding, the dataset expanded from 31 to 44 columns (43 input features plus the target), with zero text columns remaining.

Problem 4 — Features on Wildly Different Scales

PROBLEM — UNSCALED FEATURES DISTORT DISTANCE- AND GRADIENT-BASED MODELS

MonthlyRate spans into the tens of thousands, while ordinal ratings such as JobSatisfaction span only 1 to 4. For a distance-based comparison, a several-thousand unit gap in MonthlyIncome completely swamps a 3-point gap in JobSatisfaction. Gradient-based optimisation (Neural Network, Logistic Regression) also converges far more slowly and less reliably when input magnitudes differ by orders of magnitude.

SOLUTION — STANDARDISATION (STANDARDSCALER), APPLIED AFTER THE SPLIT

Technique: StandardScaler, which rescales every column to mean 0 and standard deviation 1 via $z = (x - \text{mean}) / \text{std}$. Standardisation was chosen over min-max normalisation because several columns are skewed and contain genuine outliers — min-max would squeeze the normal range of values into a tiny band based on a single extreme value.

Important: the scaler is fitted only *after* the train/test split, and only on the training set (Section 4). Fitting it on the full dataset first would let statistics from the test rows leak into training — a form of data leakage that would produce an optimistic and dishonest test score.

Pre-processing Summary

Step	Result
Original shape	(1677, 35)
Useless columns removed	4
Invalid values fixed by median imputation	4
Illogical rows capped	3
Categorical columns encoded	7 (3 methods)
Missing values remaining	0
Final input feature count	43
Feature scaling	Applied after splitting (Section 4)

4. Dataset Splitting

The cleaned and encoded data was separated into 43 input features (X) and the target (y), then split into training and test sets using an **80% / 20% stratified** split.

A stratified split (`stratify=y`) was used deliberately, not as a default choice. The minority (Left) class has only 200 rows in total; a plain random split could easily hand the test set a noticeably different class ratio than the training set, making the reported test scores unreliable. Stratification forces both sets to preserve the original 88.07% : 11.93% proportion.

Set	Rows	Percentage	Class 0 (Stayed)	Class 1 (Left)
Full dataset	1,677	100%	88.07%	11.93%
Training set	1,341	80%	88.07%	11.93%
Test set	336	20%	88.10%	11.90%

Feature scaling (Section 3, Problem 4) was then applied: the `StandardScaler` was fit only on the training set and used to transform both sets, avoiding any leakage of test-set statistics into training.

Rebalancing the Training Set

Since the training data inherits the full dataset's 88:12 imbalance (1,181 "Stayed" vs. 160 "Left" rows), training directly on it would let a model achieve high accuracy simply by always predicting "Stayed". To counter this, the minority class in the **training set only** was rebalanced using **random oversampling**: minority rows were duplicated (with replacement) until both classes matched in size, and the result was shuffled. The untouched **test set was deliberately left imbalanced**, so that it continues to reflect the real-world class distribution the models will actually be judged against.

Training Set	Class 0 (Stayed)	Class 1 (Left)
Before balancing	1,181	160
After balancing (oversampled)	1,181	1,181
Test set (left untouched)	296	40

5. Model Training & Testing

Three supervised classifiers were trained and evaluated — a Neural Network (as required), plus two additional models chosen deliberately rather than arbitrarily: **Logistic Regression**, because it turned out to be the strongest overall performer, and **Naive Bayes**, because it takes a fundamentally different (probabilistic) approach and achieves the highest recall of the three. The same data was then separately explored with **K-Means clustering** as an unsupervised learning exercise.

Model	Type
Logistic Regression	Linear
Naive Bayes	Probabilistic
Neural Network (MLP)	Deep learning

5.1 Logistic Regression

Trained with `max_iter=2000` (raised from the default of 100, since 43 features require more solver iterations to converge). Each fitted coefficient represents a feature's weight in log-odds space: a positive weight pushes the prediction toward "left", a negative weight pushes it toward "stayed".

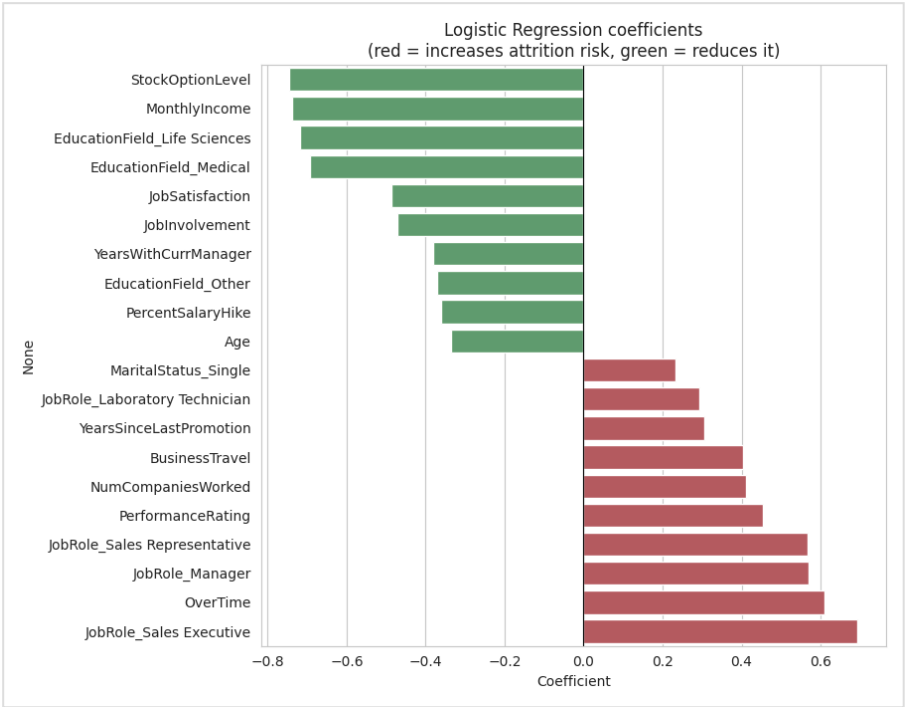


Figure 5.1 — Logistic Regression coefficients. Red bars increase attrition risk; green bars reduce it.

Strongest risk factors	Coefficient	Strongest retention factors	Coefficient
JobRole_Sales Executive	+0.694	StockOptionLevel	-0.745
OverTime	+0.610	MonthlyIncome	-0.739
JobRole_Manager	+0.571	EducationField_Life Sciences	-0.717

JobRole_Sales Representative	+0.566	EducationField_Medical	−0.694
PerformanceRating	+0.455	JobSatisfaction	−0.488

5.2 Naive Bayes

A Gaussian Naive Bayes classifier was used, appropriate since most features are continuous after encoding. Its core assumption is that every feature is conditionally independent of every other — an assumption clearly violated here, since `JobLevel` and `MonthlyIncome` correlate at 0.91 (Section 2.5). This is directly relevant to its observed behaviour: see Section 6 and Section 7 for how this assumption shapes its results.

5.3 Neural Network (Multi-Layer Perceptron)

A feed-forward network with architecture `43 → 64 → 32 → 1` was trained using scikit-learn's `MLPClassifier`:

- **Activation:** ReLU on both hidden layers, avoiding vanishing gradients.
- **Optimiser:** Adam, an adaptive optimiser that performs well without extensive manual tuning.
- **Regularisation:** L2 penalty (`alpha=0.001`) to reduce overfitting.
- **Early stopping:** 15% of the training data is automatically held back as an internal validation set; training stops if validation accuracy does not improve for 15 consecutive epochs (`n_iter_no_change=15`), rather than always running the full 300-epoch cap.

In practice, early stopping triggered genuinely: training halted after just **37 epochs** out of a possible 300. The plots below show the training loss and the internal validation-set accuracy recorded after every single epoch up to that point.

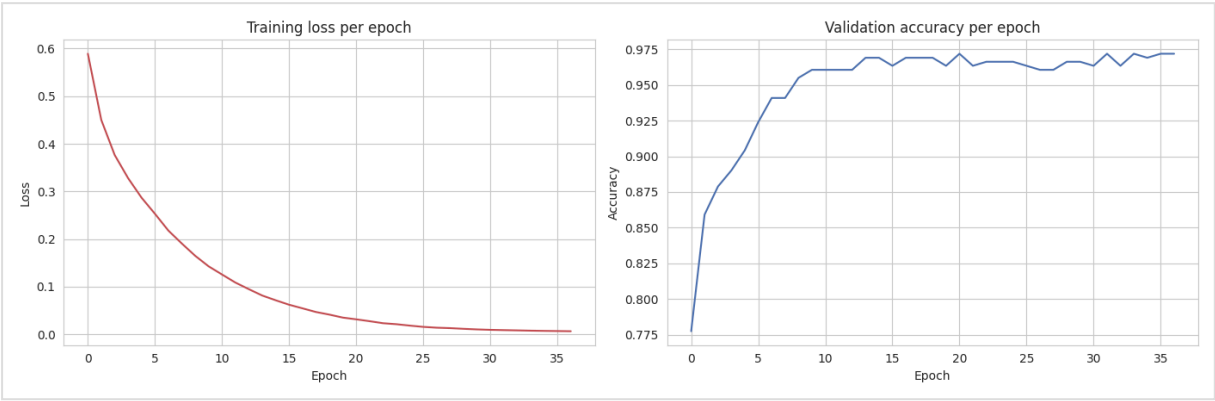


Figure 5.2 — Neural network training loss (left) and validation-set accuracy (right), tracked per epoch until early stopping triggered at epoch 37.

5.4 Unsupervised Approach — K-Means Clustering

Here the labels were deliberately set aside to ask a different question: without being told who left, does K-Means discover natural groups in the employee data, and do those groups happen to align with attrition? All 43 scaled features (excluding the target) were used.

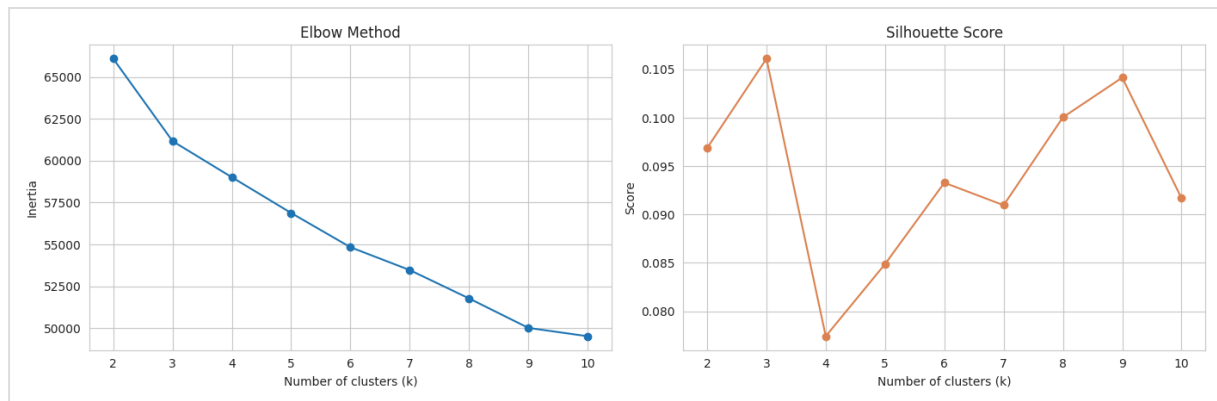


Figure 5.3 — Elbow method (inertia) and silhouette score for $k = 2$ through 10.

The elbow and silhouette diagnostics did not point to a strongly separated cluster structure (silhouette scores stayed below 0.11 across every k tested), which is itself informative — the data does not contain sharply distinct natural groups. $k = 2$ was fixed deliberately so the two clusters could be compared directly against the two real Attrition classes.

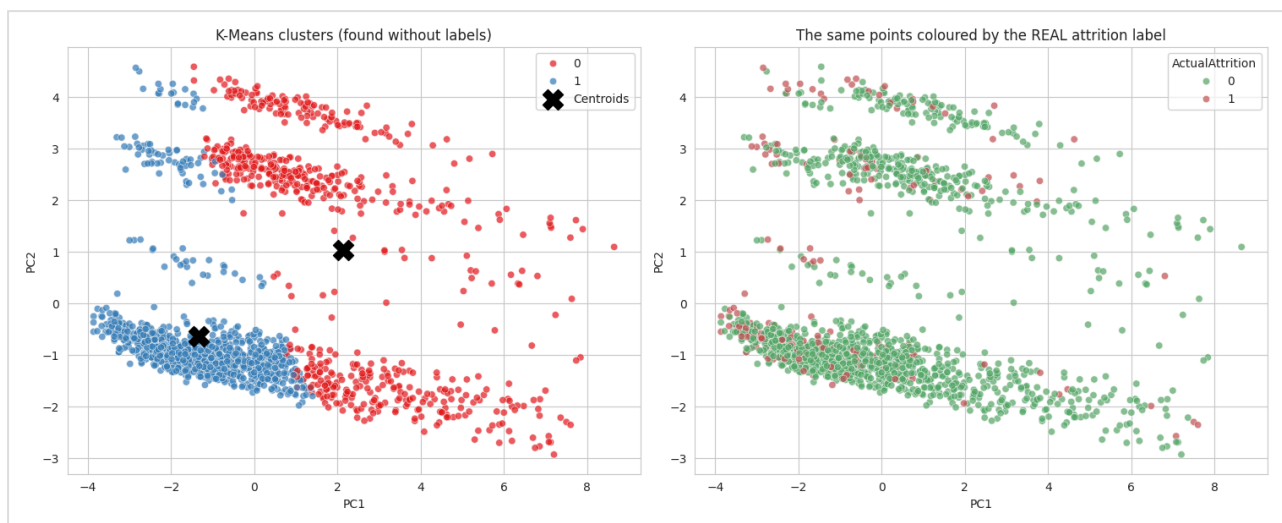


Figure 5.4 — K-Means clusters (left) vs. the same points coloured by the real Attrition label (right), visualised via PCA (2 components explain 20.6% of total variance).

	Actual: Stayed (0)	Actual: Left (1)	Cluster Size	Attrition Rate
Cluster 0	593	53	646	8.20%
Cluster 1	884	147	1,031	14.26%

Adjusted Rand Index (1.0 = perfect match with the real labels, 0.0 = random): **-0.0175**.

WHAT K-MEANS ACTUALLY FOUND

K-Means did **not** rediscover attrition — the Adjusted Rand Index near zero shows the clusters are essentially unrelated to the real labels. Instead, it split the workforce along **seniority**, the strongest source of variance in the feature space:

Average of	Cluster 0 (Senior)	Cluster 1 (Junior)
Age	39.48	33.88
MonthlyIncome	9,716.39	3,990.29
TotalWorkingYears	15.65	7.58
JobLevel	2.89	1.44
YearsAtCompany	10.24	4.62

This is still a useful finding: the junior cluster's attrition rate (14.26%) is noticeably higher than the senior cluster's (8.20%), so seniority is genuinely linked to attrition — just not strongly enough to be the dominant explanation on its own. More broadly, this illustrates that unsupervised learning finds whatever structure dominates the feature space, not necessarily the structure a practitioner happens to be interested in. Since attrition is a weak, scattered signal spread across many features (Section 2.5), K-Means could not recover it — which is exactly why the supervised models, explicitly shown the labels during training, perform meaningfully better at this specific task.

6. Model Selection / Comparison Analysis

Model	Accuracy	Precision	Recall	F1-Score	AUC
Logistic Regression	0.7560	0.2717	0.6250	0.3788	0.7606
Naive Bayes	0.5685	0.1818	0.7500	0.2927	0.6990
Neural Network	0.8452	0.3500	0.3500	0.3500	0.7048

Majority-class baseline (always predicting "Stayed"): **0.8810** accuracy. Every model must be judged against this number, not against zero.

6.1 Prediction Accuracy Comparison

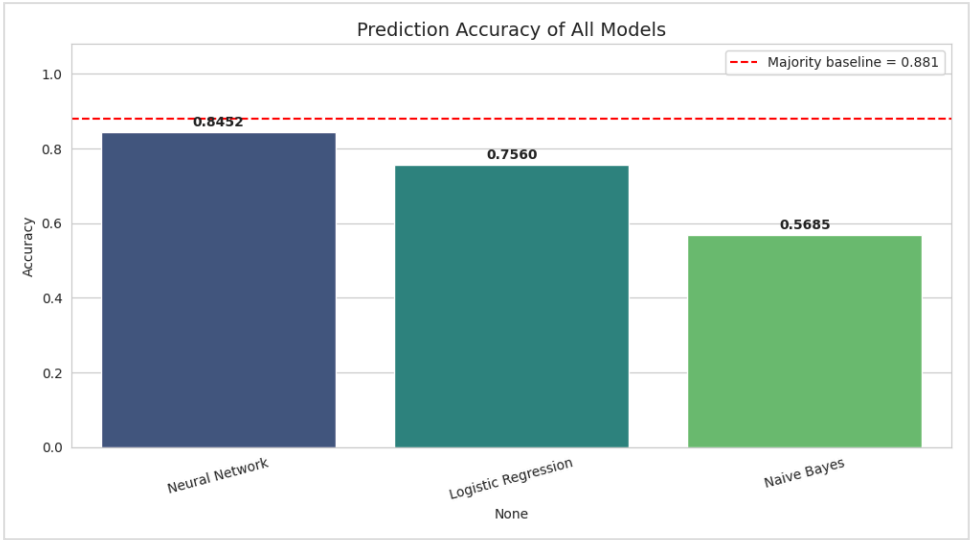


Figure 6.1 — Test-set accuracy of all three models, with the majority-class baseline marked.

The Neural Network achieves the highest raw accuracy (0.8452), close to the 0.8810 baseline, while Naive Bayes falls well below the baseline (0.5685). As discussed below, raw accuracy is a misleading metric on this imbalanced dataset and should not be read in isolation.

6.2 Precision and Recall Comparison

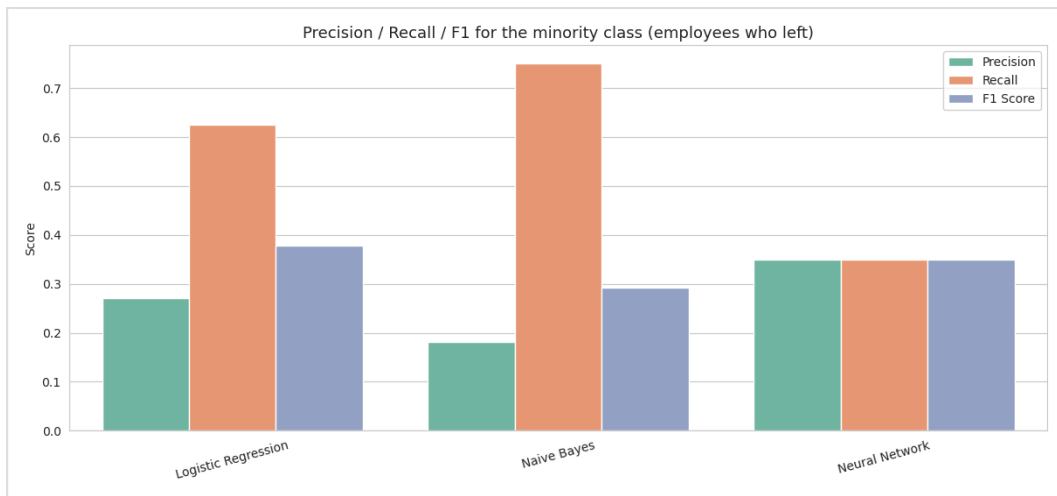


Figure 6.2 — Precision, recall, and F1-score for the minority class (employees who left), across all three models.

Precision answers: of all employees the model flagged as leaving, how many really left? Low precision means HR effort is wasted on employees who were never actually going to leave. **Recall** answers: of all employees who really left, how many did the model catch? Low recall means the model misses real leavers — the more expensive failure for a company, since a missed leaver costs a full replacement hire while a false alarm only costs a conversation. For this project, recall is therefore the more business-relevant metric of the two.

6.3 Confusion Matrices

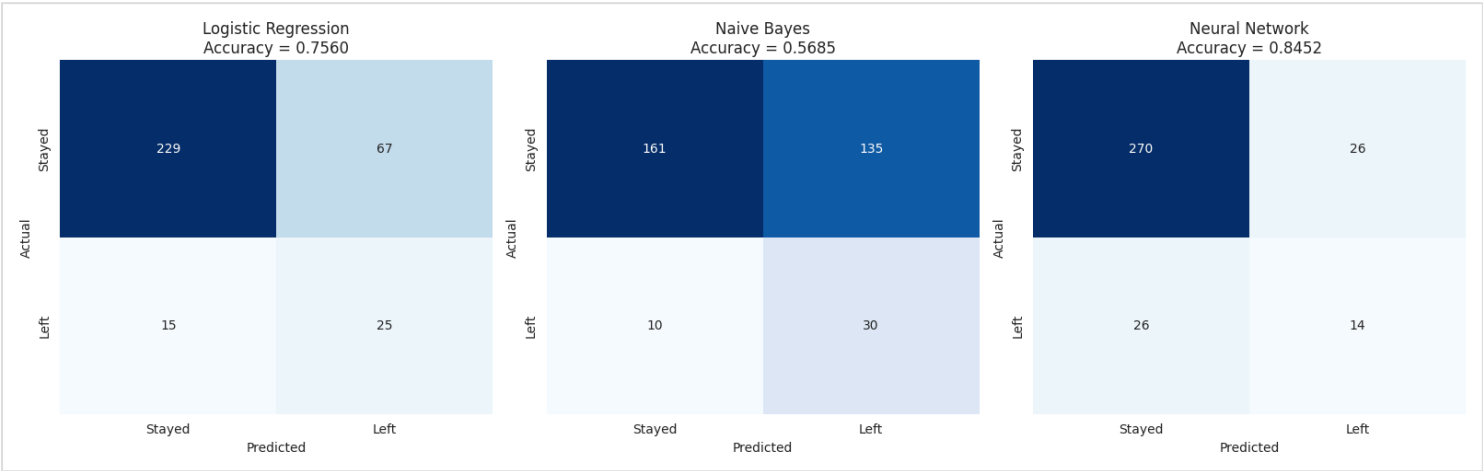


Figure 6.3 — Confusion matrices for all three models on the untouched test set (n = 336).

Model	True Neg.	False Pos.	False Neg.	True Pos.	Leavers Caught
Logistic Regression	229	67	15	25	25 / 40 (62.5%)
Naive Bayes	161	135	10	30	30 / 40 (75.0%)
Neural Network	270	26	26	14	14 / 40 (35.0%)

6.4 ROC Curve and AUC Score

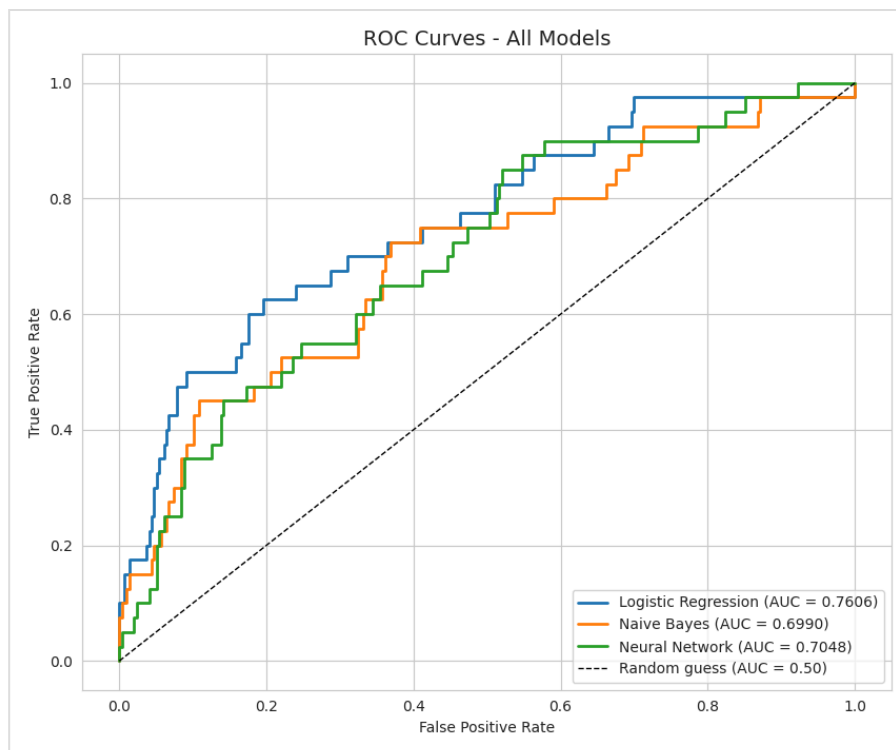


Figure 6.4 — ROC curves for all three models against the random-guess diagonal.

AUC is the fairest single number for this imbalanced problem: it does not depend on the 0.5 decision threshold and is not inflated by the large majority class. It measures how well a model ranks a random leaver above a random stayer.

Model	AUC
Logistic Regression	0.7606
Neural Network	0.7048
Naive Bayes	0.6990

6.5 Best Model per Metric

Metric	Best Model	Score
Accuracy	Neural Network	0.8452
Precision	Neural Network	0.3500
Recall	Naive Bayes	0.7500
F1-Score	Logistic Regression	0.3788
AUC	Logistic Regression	0.7606

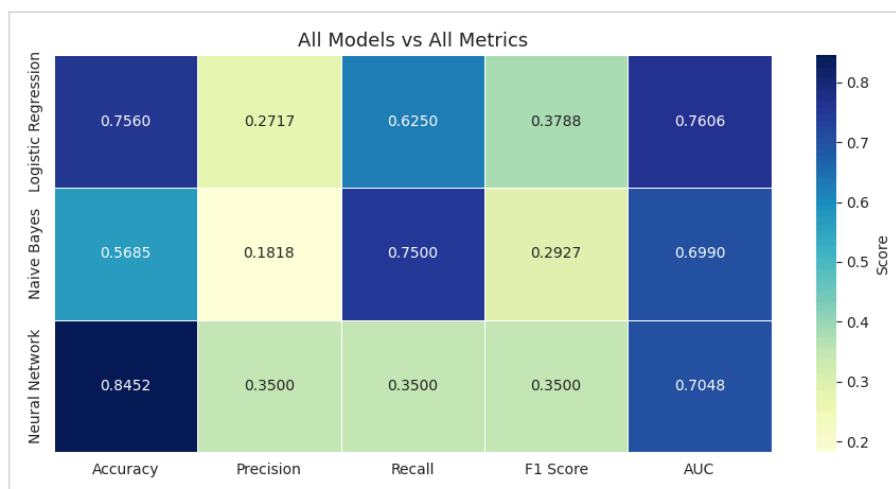


Figure 6.5 — Heatmap summary of all models across all five metrics.

6.6 The Accuracy Paradox: Imbalanced vs. Balanced Training

To directly justify the oversampling step in Section 4, every model was also trained on the original, *unbalanced* training data for comparison.

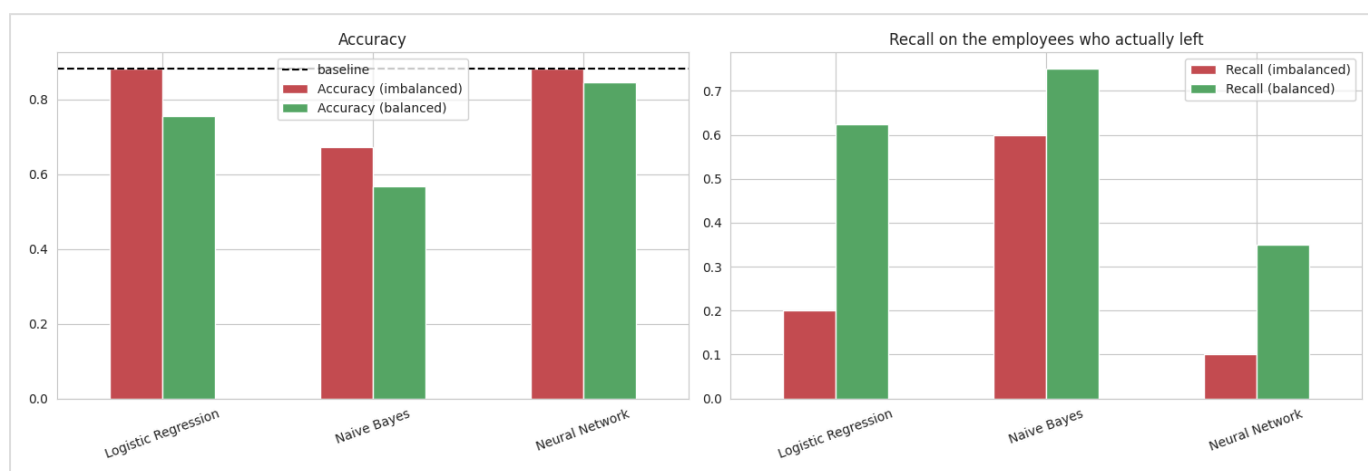


Figure 6.6 — Accuracy and recall when trained on imbalanced vs. balanced training data.

Model	Acc. (imbalanced)	Acc. (balanced)	Recall (imbalanced)	Recall (balanced)
Logistic Regression	0.8810	0.7560	0.200	0.625
Naive Bayes	0.6726	0.5685	0.600	0.750
Neural Network	0.8810	0.8452	0.100	0.350

Trained on the raw imbalanced data, both Logistic Regression and the Neural Network score accuracy right at the 0.881 baseline — but their recall collapses to 0.20 and 0.10 respectively. Those models are barely catching any real leavers at all; they are simply predicting "stayed" almost every time and collecting free accuracy from the 88% majority class. After oversampling, accuracy drops but recall rises sharply across every model — the models are now actually attempting the task that matters. This is the **accuracy paradox**, and it is the single most important pattern to understand about this dataset.

7. Conclusion

7.1 What We Understand From the Results

Metric	Winner	Score
Best AUC	Logistic Regression	0.7606
Best F1	Logistic Regression	0.3788
Best Recall	Naive Bayes	0.7500
Best Accuracy	Neural Network	0.8452
Baseline to beat	0.8810 (accuracy)	

1. Attrition in this dataset is genuinely hard to predict. The correlation test in Section 2 already warned of this: no feature correlated with the target beyond about 0.20, so a very high accuracy result was never realistic here.
2. No single model wins on every metric, and that is the most interesting finding of the comparison. Logistic Regression gives the best overall ranking ability (AUC and F1), Naive Bayes catches the most real leavers (recall), and the Neural Network achieves the highest accuracy by being the most conservative in what it flags.
3. Accuracy is the wrong headline metric for this problem. A model can reach 0.881 accuracy by never predicting attrition at all, so AUC, recall, and F1 are judged as the more meaningful metrics instead (Section 6.6).
4. The unsupervised K-Means result reinforces the same conclusion: without labels, the dominant structure discoverable in the data is seniority, not attrition.

7.2 Comments on Each Model's Performance

Logistic Regression — the most reliable model overall

Best AUC and best F1. Attrition risk in this dataset appears to be built from many small, additive effects — a bit of overtime here, slightly low pay there, no stock options — and a linear model is well suited to summing exactly that kind of weak, distributed evidence. It is also the easiest model to explain to an HR audience, since each coefficient reads directly as a risk or retention factor (Section 5.1).

Naive Bayes — highest recall, lowest precision

It flags a very large share of the workforce as at-risk, so it catches the most real leavers (75.0% recall) but at the cost of the most false alarms, which is why its accuracy is the lowest of the three. The likely cause is its independence assumption: it treats `JobLevel` and `MonthlyIncome` as unrelated when they in fact correlate at 0.91 (Section 2.5), so it effectively double-counts the same evidence and becomes overconfident in flagging risk.

Neural Network — highest accuracy, but conservative

The MLP achieves the highest accuracy and a competitive AUC, but is noticeably more conservative in what it flags as "Left" — its recall (0.350) is the lowest of the three. With roughly 2,360 balanced training rows and 43 input features, the network has relatively little data per parameter, and training halted early at just 37 epochs (Section 5.3). Deep learning approaches typically need substantially more examples than this to show a clear advantage over simpler models.

The pattern across all three

Each model sits at a different point on the same precision/recall trade-off. Naive Bayes catches the most leavers while raising the most false alarms; the Neural Network does the opposite; Logistic Regression sits between them with the best overall balance. None of the three escapes the trade-off entirely, because the two classes genuinely overlap in feature space (Section 2.5).

7.3 Why We Are Getting These Results

1. **Weak feature signal.** The strongest correlation with the target was only about 0.19–0.20 in magnitude. The available features describe an employee's measurable situation, but the actual decision to resign also depends on factors no HR table records — a difficult manager, a family relocation, a competing offer, or simple burnout. These causes are invisible to every model used here.
2. **Severe class imbalance.** Only 200 of 1,677 employees left (11.93%). Section 6.6 demonstrated the direct consequence: without rebalancing, models simply learn to predict "stayed" for almost everyone.
3. **A modestly sized dataset relative to feature count.** 1,677 rows and 43 features after encoding is a workable size for linear and probabilistic models, but limits how much a Neural Network with thousands of internal weights can meaningfully learn beyond them.
4. **The accuracy/recall trade-off is unavoidable here.** Because the two classes overlap heavily in feature space, catching more leavers always means raising more false alarms. Which point on that trade-off is "best" depends on business cost rather than mathematics — if replacing an employee costs far more than an extra check-in conversation, a higher-recall model such as Naive Bayes becomes the more defensible choice despite its lower accuracy.
5. **Keeping the correlated features was validated, not just assumed.** As shown in Section 2.5, removing the redundant features left two of the three models unchanged or worse, confirming that the weak, broadly distributed signal in this dataset benefits from retaining the full feature set rather than trimming it.

7.4 Challenges Faced

1. **Missing values that did not look missing.** `isnull()` returned zero, so the dataset initially appeared clean. The real errors were valid-looking numbers, such as `Education = 15` on a 1–5 scale. Finding them required a manual range check against each column's documented scale, which was the most time-consuming part of pre-processing.
2. **Choosing the right encoding per column.** A single encoder applied to every text column would have implied a false numeric order between categories such as job roles. Separating the columns into binary, ordinal, and nominal groups (Section 3, Problem 3) required deliberate thought about each column individually.
3. **The imbalance trap.** Early models scored a superficially excellent 88% accuracy, but inspecting the confusion matrix revealed they were catching almost no real leavers at all. Realising that accuracy was masking total failure on the class that

actually matters reshaped the direction of the whole analysis.

- 4. **Avoiding data leakage.** Getting the processing order right mattered: split first, then fit the scaler on the training set only, then oversample the training set only. Doing scaling or oversampling before the split would have leaked test-set information into training and produced inflated, dishonest scores.
- 5. **Correlation alone was misleading.** The correlation test gave `JobRole` a score of roughly 0.03, suggesting near-irrelevance, while the EDA plots in Section 2.7 showed Sales Representatives leaving at over 33%. Correlation only measures straight-line relationships on ordered numbers and could not see this categorical effect at all — both tools were needed together.
- 6. **Accepting a modest result.** The hardest part was accepting that an AUC around 0.76 is a legitimate, honest outcome for this dataset rather than a flaw in the approach. Chasing a noticeably higher number would most likely have meant overfitting to the test set.

7.5 Final Summary

Dataset	1,677 employees, 35 original columns, binary classification target, imbalanced at roughly 88:12.
Pre-processing	Dropped 4 useless columns, repaired 4 invalid values via median imputation, capped 3 logically impossible rows, encoded 7 categorical columns using three different methods, standardised all features after splitting.
Models	3 supervised classifiers (Logistic Regression, Naive Bayes, Neural Network) plus K-Means clustering.
Recommended model	Logistic Regression — AUC 0.7606, F1 0.3788. Offers the best overall balance between catching real leavers and remaining explainable, which matters when results need to be justified to an HR team.
Practical takeaway for HR	The strongest and most actionable risk factors identified were overtime, low monthly income, the absence of stock options, low job level, and short tenure. Even an imperfect model surfaces directly actionable findings.
Possible improvements	SMOTE in place of simple duplication for oversampling, ensemble methods such as Random Forest or XGBoost, engineered ratio features (e.g. income per year of experience), hyperparameter tuning via GridSearchCV, and adjusting the decision threshold away from 0.5 to match the real business cost of a missed leaver.